

Module 6 - Deploy Anywhere (Dockerised Microservices)

Ryan Gogerty - JHU Modern Concepts in Python, Module 6

1. Docker Compose Architecture

The stack is defined entirely in `docker-compose.yml` and comes up on a clean machine with `'docker compose up --build'`. Four services share Compose's default private network and address each other by service name. `'db'` is a PostgreSQL 16 container with a named volume (`pgdata`) for durable storage and an `init.sql` bootstrap mounted into `/docker-entrypoint-initdb.d`. `'rabbitmq'` is the message broker (RabbitMQ 3 with the management plugin), exposing AMQP on 5672 and the management UI on 15672. `'web'` is the Flask front end, built from `src/web/Dockerfile` and published on `localhost:8080`. `'worker'` is the Python consumer, built from `src/worker/Dockerfile`, with the data directory bind-mounted read-only at `/data`. Both application images are built from the `./src` context so they can share one `'db'` package (a single connection helper and JSON loader), install only their own pinned requirements, and run as a non-root user (`USER 1000`). Health checks on `db` and `rabbitmq` gate the app containers via `depends_on`: `condition: service_healthy`, so nothing starts talking to a broker or database that is not ready yet.

2. Service Roles and Why the Tiers Are Split

Decoupling the web tier from data-modifying work improves reliability, scalability, and security. The web app stays fast and stateless: it renders analytics and, when a button is clicked, publishes a task and immediately returns HTTP 202 - it never waits on a scrape or a heavy recompute. The worker owns all long-running and write operations, so it can be scaled, retried, and rate-limited independently. Routing every write through the queue gives buffering and backpressure (no request timeouts under load), durable at-least-once delivery, and clear operational visibility through the broker's queue metrics. This mirrors the classic producer/consumer and least-privilege patterns.

3. RabbitMQ Message Flow

The web tier's `publisher.py` opens a channel (pika `BlockingConnection` using `RABBITMQ_URL`) and idempotently declares a durable direct exchange named `'tasks'`, a durable queue named `'tasks_q'`, and a binding on routing key `'tasks'`. `publish_task(kind, payload)` sends a compact JSON body `{kind, ts, payload}` with `delivery_mode=2` (a persistent message that survives a broker restart) and closes the connection in a finally block; any failure is raised so the Flask endpoint can return 503 rather than silently drop work. The Pull Data button publishes kind `'scrape_new_data'`; Update Analysis publishes `'recompute_analytics'`. The worker's `consumer.py` connects with the same URL, declares the same durable topology, and sets `basic_qos(prefetch_count=1)` so it processes exactly one message at a time - backpressure that prevents a slow task from letting the queue overwhelm the worker.

4. Database Initialisation, Migration, and SQL Safety

Schema creation happens two ways, both idempotent. On a clean machine, Postgres runs

Module 6 - Deploy Anywhere (Dockerised Microservices)

Ryan Gogerty - JHU Modern Concepts in Python, Module 6

src/db/init.sql from its init directory, creating the applicants table, the ingestion_watermarks table, and the analysis_summary cache. Independently, the worker calls db.load_data.create_schema() at start-up, so the schema also exists in non-Docker runs (local dev and CI). load_data.py then bulk-loads data/applicant_data.json with parameterised, sql.SQL-composed INSERTs and ON CONFLICT (url) DO NOTHING, so re-running never duplicates rows. Every statement across the codebase binds values through placeholders and quotes identifiers with sql.Identifier - runtime input can never alter a query's shape.

5. Worker Behaviour: Idempotent, Watermarked, Transactional

consumer.on_message parses each JSON message and routes by 'kind' through a task map to handle_scrape_new_data or handle_recompute_analytics. Each message runs in its own transaction (a connection opened per message that commits only on clean exit); the message is acknowledged only after that commit. On any handler error the transaction rolls back and the message is basic_nack'd with requeue=False, so a malformed or poison task is dropped rather than looping forever. handle_scrape_new_data reads the high-water mark from ingestion_watermarks, fetches only records with a larger Grad Cafe result id, inserts them idempotently, and advances the watermark to the max id seen - so repeated Pull Data clicks add only genuinely new rows. handle_recompute_analytics recomputes the cached scalar metrics into analysis_summary; the web page shows those cached values and their 'last recomputed' timestamp, which is how the asynchronous recompute becomes visible in the UI.

6. Build, Run, and Verification Steps

1) From module_6/, run 'docker compose up --build'. 2) Wait for the db and rabbitmq health checks to pass; the worker then seeds the base data and recomputes the cache. 3) Open <http://localhost:8080> to see the analytics page. 4) Open <http://localhost:15672> (guest/guest, dev only) to see the 'tasks_q' queue. 5) Click Pull Data - the app returns a queued banner (202); watch a message flow through tasks_q and the live entry count rise after the worker commits. 6) Click Update Analysis - the cached summary and its timestamp refresh. The screenshots below capture the running website and the RabbitMQ management UI.

7. Container Registry (Docker Hub)

Both images are published to a public Docker Hub repository named module_6. Build and push with, for example: 'docker build -t rgogerty/module_6:web-v1 -f src/web/Dockerfile src' and 'docker build -t rgogerty/module_6:worker-v1 -f src/worker/Dockerfile src', then 'docker login' and 'docker push rgogerty/module_6:web-v1' (and the worker tag). The public repository is at https://hub.docker.com/r/rgogerty/module_6, and pull/run instructions and the live registry link are in the README.

Module 6 - Deploy Anywhere (Dockerised Microservices)

Ryan Gogerty - JHU Modern Concepts in Python, Module 6

8. Quality Gates: Pylint and Pytest

A dedicated Module 6 GitHub Actions workflow runs on changes under `module_6/`. The lint job installs the service requirements plus pylint and scores the source at 10.00/10 (`--fail-under=10`). The test job spins up a Postgres service container and runs pytest with `--cov-fail-under=100` over `src/`, so the 39-test suite must pass at 100% coverage. The Module 5 workflows are scoped to `module_5/` paths and do not interfere.

Module 6 - Deploy Anywhere (Dockerised Microservices)

Ryan Gogerty - JHU Modern Concepts in Python, Module 6

9. Required Screenshots

Module 6 - Deploy Anywhere (Dockerised Microservices)

Ryan Gogerty - JHU Modern Concepts in Python, Module 6

Grad Café Data Analysis

Pull DataUpdate Analysis

Analysis

Analysis of 25,500 graduate school applicant entries. Live counts update the instant the worker commits a scrape. Last rendered: July 01, 2026 at 02:48 AM

Cached Summary (refreshed asynchronously by the worker)

Last recomputed: July 01, 2026 at 02:46:45 AM — click Update Analysis to enqueue a refresh.

TOTAL APPLICANTS (CACHED)
Answer: 25500

DISTINCT UNIVERSITIES
Answer: 1222

ACCEPTANCE RATE (CACHED)
Answer: 37.08%

AVG GPA (CACHED)
Answer: 3.784

Q1 — FALL 2026 ENTRIES
Answer: 25,160

Q2 — INTERNATIONAL APPLICANTS
Answer: 44.24%

Q5 — FALL 2026 ACCEPTANCE RATE
Answer: 36.90%

Q3 — AVG GPA (REPORTERS)
Answer: 3.78

Q3 — Average Academic Metrics (among applicants who reported each)

Metric	Answer
GPA	Answer: 3.78
GRE (quant / combined)	Answer: 263.95
GRE Verbal	Answer: 160.96
GRE Analytical Writing	Answer: 4.4

Q10 — Top Programs by Acceptance Rate (min 10 applicants)

Program	Answer: Acceptance Rate	Accepted / Total
Business Analytics	Answer: 91.7%	11 / 12
Optics and Photonics	Answer: 90.9%	10 / 11
International Development	Answer: 90.0%	9 / 10
Biotechnology	Answer: 85.7%	12 / 14
Environmental Management	Answer: 84.6%	11 / 13

Q11 — Average GPA by Admission Outcome (GPA reporters only)

Outcome	Answer: Avg GPA	n
Waitlisted	Answer: 3.825	1,202
Rejected	Answer: 3.804	6,856
Accepted	Answer: 3.792	6,301

Data sourced from The Grad Café · Ryan Gogerty · JHU Software Concepts Module 6 (Deploy Anywhere)

Module 6 - Deploy Anywhere (Dockerised Microservices)

Ryan Gogerty - JHU Modern Concepts in Python, Module 6

Figure 1 - The running web app at http://localhost:8080 (Grad Cafe analytics).

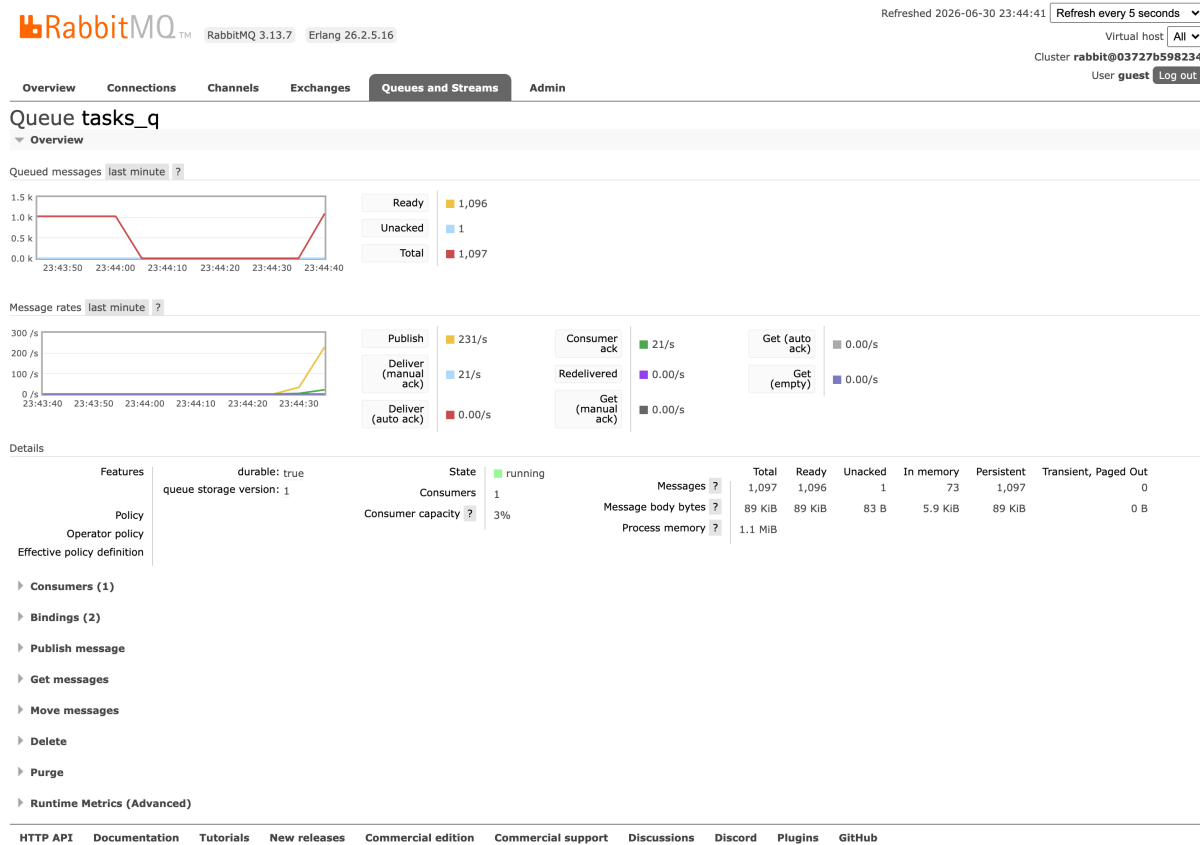


Figure 2 - The RabbitMQ management UI at http://localhost:15672 (tasks_q queue).